efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

2024 / 2025

Les tableaux & chaînes de caractères

(basé sur le cours d'Asma Gabis)

Rado Rakotonarivo
rado.rakotonarivo@efrei.fr

EESPIG
CONFÉRENCE DES GRANDES ÉCOLES
Cti

Plan du cours

- Les tableaux 1D
- Les chaînes de caractères
- Les tableaux 2D

Les tableaux 1D

Les tableaux 1D

Motivation

Problème : Écrire un algorithme qui permet de calculer la moyenne d'un groupe de 3 étudiants .

Solution :

1. Déclarer 3 variables (une pour chaque étudiant) de type réel : *moy_et1*, *moy_et2*, *moy_et3*
2. Déclarer une variable de type réel pour contenir la moyenne générale : `moy_gen`
3. Demander à l'utilisateur de saisir chacune des 3 valeurs
   - `Lire(moy_et1)`
   - `Lire(moy_et2)`
   - `Lire(moy_et3)`
4. Calculer la moyenne générale :
   - `moy_gen ← (moy_et1 + moy_et2 + moy_et3) / 3`

Et si le nombre d'étudiants est égal à 100 ?

Les tableaux 1D

Motivation

Problème : Écrire un algorithme qui permet de calculer la moyenne générale d'un groupe de 100 étudiants .

Solution potentielle :

1. Déclarer 100 variables (une pour chaque étudiant) de type réel : *moy_et1*, *moy_et2*, *moy_et3*, … *moy_et100*
2. Déclarer une variable de type réel pour contenir la moyenne générale : `moy_gen`
3. Demander à l'utilisateur de saisir chacune des 100 valeurs
   - `Lire(moy_et1)`
   - `Lire(moy_et2)`
   - …
   - `Lire(moy_et100)`
4. Calculer la moyenne générale :
   - `moy_gen ← (moy_et1 + moy_et2 + moy_et3 + … + moy_et100) / 100`

LABORIEUX

Et si le nombre d'étudiants n'est pas connu à l'avance ?

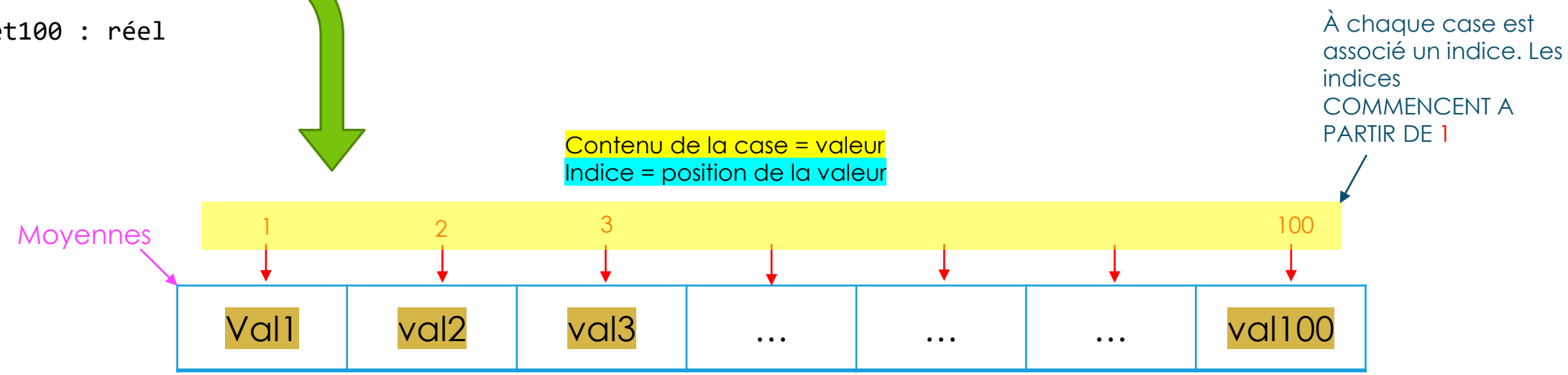
Les tableaux 1D

Les tableaux : La solution dans ce cas

En algorithmique, il est possible de rassembler un ensemble de valeurs sous un seul nom.

La structure qui regroupe ces valeurs est appelée : Tableau

```
Moy_et1 : réel
Moy_et2 : réel
Moy_et3 : réel
…
Moy_et100 : réel
```



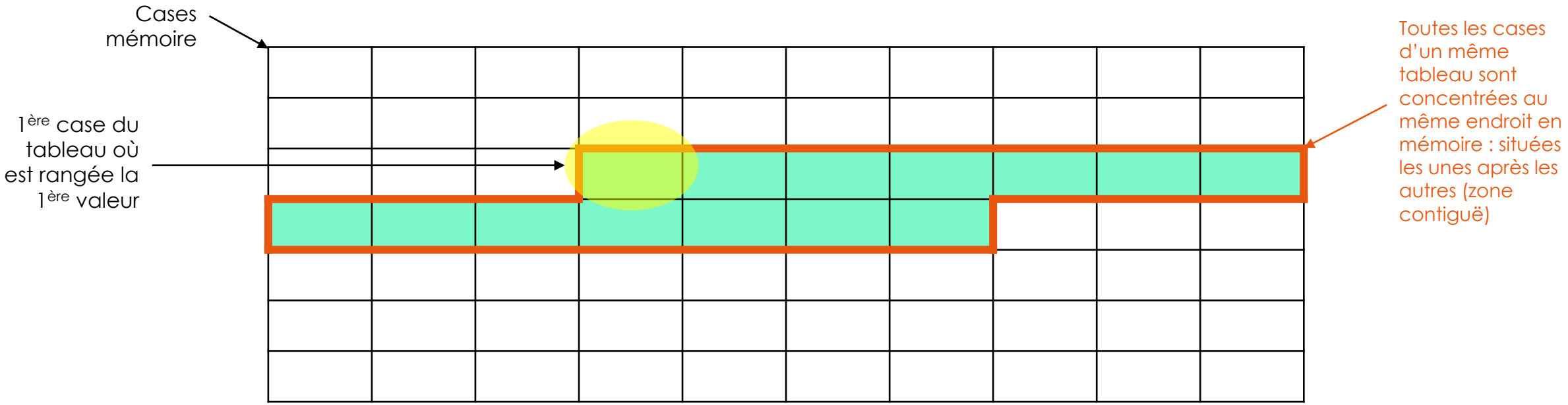
Les tableaux 1D

Définition

Un tableau est une collection de données de même type référencées par un nom commun

Comparé à un type de base (entier, réel, caractère, …), c'est un type agrégé ou dérivé.

Les éléments du tableau occupent des emplacements mémoire contigus.



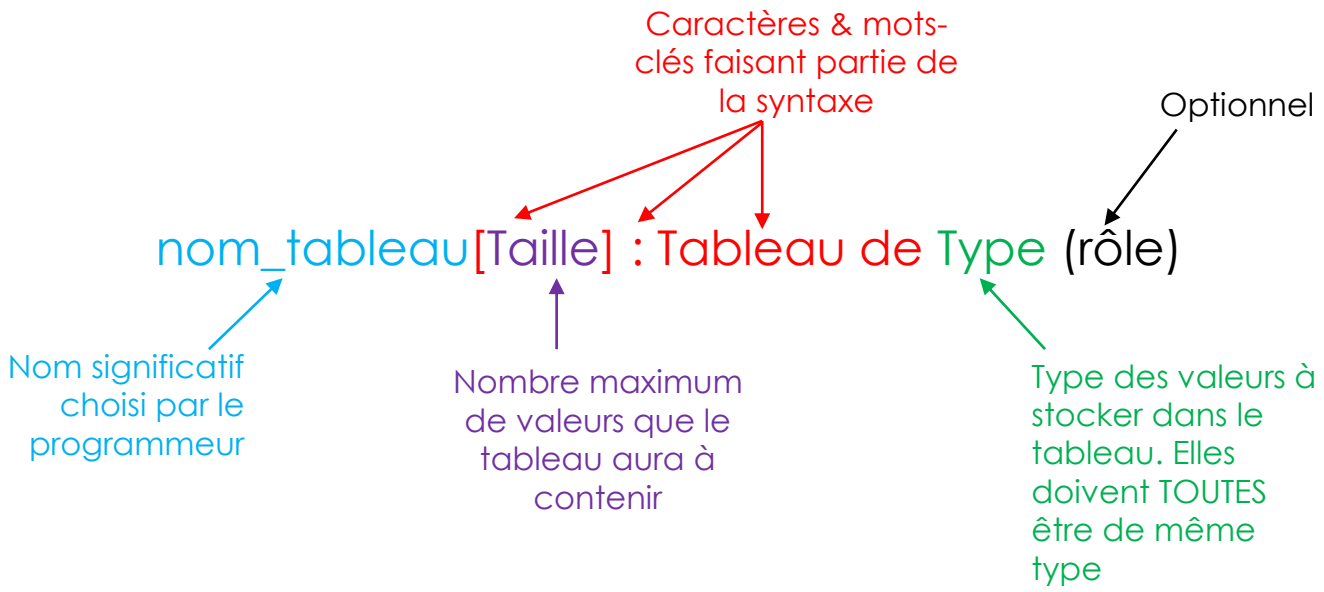
Les tableaux 1D

Déclaration d'un tableau

Un tableau se déclare comme une variable. Il est donc nécessaire de :

- Lui attribuer un nom (comme pour toute variable d'un type de base)
- Préciser son type (le type des valeurs qu'il va stocker)
- Préciser le nombre de valeurs qu'il stockera au maximum

Syntaxe :



Les tableaux 1D

Déclaration d'un tableau

Exemple :

```
Algorithme Exemple_declaration_tableaux


Variables :
         moyennes[496] : Tableau de réel (moyennes de 496 étudiants)
         departements[100]: Tableau de entier (numéros des départements français)
         alphabets[26]: Tableau de caractère (alphabet de la langue française)
         Tab1[50], Tab2[32] : Tableau de entier

Début
      ...
Fin
```

Cas de déclaration de deux tableaux différents avec des noms et des tailles différentes mais qui ont le même type de valeurs.

Les tableaux 1D

Accès aux valeurs d'un tableau

- Chaque case du tableau est caractérisée par :
  - Un indice : représentant sa position dans le tableau et commençant par convention à la valeur 1
  - Un contenu : représentant la valeur qui y est stockée.
- L'accès à une valeur se fait comme suit : nom_tableau[indice]

Mettre la position de la valeur souhaitée.
Il peut être une valeur, une variable ou une expression mathématique

Exemples

```
Algorithme Exemple1_acces_valeur_tableau

Variables :
          Tab[50]: Tableau de entier

Début
     Tab[1] ← 3
     Lire(Tab[2])
     Ecrire (Tab[1])
Fin
```

```
Algorithme Exemple2_acces_valeur_tableau

Variables :
          Tab[50]: Tableau de entier
           i : entier
Début
     i ← 1
     Tab[i] ← 3
     Tab[i+1] ← 4
     Ecrire (Tab[i])
Fin
```

Les tableaux 1D

Initialisation d'un tableau

Pour initialiser un tableau en dur, il suffit de lui affecter des valeurs fournies entre accolades {}

nom_tableau ← {val1, val2, … }

Exemple

```
Algorithme Exemple_initialisation_tableau

Variables :
          Tab[5]: Tableau de entier

Début
     Tab ← {1, 2, -4, 78, 34}
Fin
```

Équivalent à

Tab[1] ← 1
Tab[2] ← 2
Tab[3] ← -4
Tab[4] ← 78
Tab[5] ← 34

Les tableaux 1D

Remplir un tableau

Il est possible de remplir un tableau par des saisies au clavier ➔ Utilisation de la boucle « Pour » et remplissage des cases une à une

Exemple

```
Algorithme remplir_tableau_saisies

Variables :
          Tab[10]: Tableau de entier
           i : entier
Début
     Pour i ← 1 à 10 Faire
           Ecrire (« Saisie de la valeur : », i)
           Lire(Tab[i])
     Fin Pour
Fin
```

L'instruction Saisir est utilisée une seule fois dans une boucle ➔ Algorithme moins long.

Les tableaux 1D

Afficher les valeurs d'un tableau

De la même façon, le seul moyen pour afficher le contenu d'un tableau est de le parcourir de la 1ère case à la dernière case et d'afficher leur contenu valeur par valeur.

Exemple

```
Algorithme remplir_afficher_tableau

Variables :
          Tab[10]: Tableau de entier
           i : entier
Début
     Pour i ← 1 à 10 Faire
          Ecrire (« Saisie de la valeur : », i)
          Lire(Tab[i])
     Fin Pour
     Ecrire (« Les valeurs du tableau sont : \NL»)
     Pour i ← 1 à 10 Faire
          Ecrire (Tab[i], «   »)
     Fin Pour

Fin
```

Ici toutes les valeurs du tableau vont s'afficher sur une même ligne séparées par des espaces

Les tableaux 1D

Taille Physique Vs Taille logique

Et si on ne connaissait pas à l'avance le nombre de valeurs à stocker dans le tableau ?

- À la déclaration d'un tableau, la taille indiquée entre crochets [] représente le nombre de cases à réserver en mémoire : C'est une estimation du nombre maximum de valeurs à stocker

➔ Elle représente la Taille Physique du tableau.

- Le nombre exact de valeurs à stocker n'est connu que lors du remplissage du tableau

➔ Il s'agit de la Taille logique

Exemple :

1	2	3	4	5	6	7	8	9	10	11	12	13	14
-12	11	13	12	23	10	21	15	32	?	?	?	?	?

Taille physique = 14
Taille logique = 9
Les valeurs significatives sont placées dans les cases d'indices 1 … 9
Les valeurs en position 10, 11, … 14 sont indéfinies , donc ignorées

Les tableaux 1D

Taille Physique Vs Taille logique

Illustration :

- Le nombre estimé d'étudiants inscrits en L1 est égal à 500 ➔ Prévoir un amphithéâtre d'une capacité égale à 500 (taille physique = 500)
- Le nombre exact d'étudiants inscrits finalement en L1 est égal à 496 ➔ Le nombre de sièges réellement occupés en amphithéâtre est 496 (taille logique = 496) : 4 sièges libres

À retenir !!!

1. La taille physique est précisée à la déclaration du tableau
2. La taille logique est connue dans le corps de l'algorithme
3. La taille logique est toujours inférieure ou égale à la taille physique

Les tableaux 1D

Taille Physique Vs Taille logique

Exemple

Saisie sécurisée pour s'assurer que le nombre de valeurs (taille logique) ne dépasse pas le nombre de cases réservées (taille physique)

```
Algorithme taille_physique_logique
Constantes:
            TPhy = 10
Variables :
            Tab[TPhy]: Tableau de entier
            i, TLog : entier
Début
      Répéter
            Ecrire (« Saisir le nombre de valeurs: »)
            Lire(TLog)
      Tant que (TLog ≤ 0 OU TLog > Tphy)
      Pour i ← 1 à TLog Faire
            Ecrire (« Saisie de la valeur : », i)
            Lire(Tab[i])
      Fin Pour
      Ecrire (« Les valeurs du tableau sont : \NL»)
      Pour i ← 1 à TLog Faire
            Ecrire (Tab[i], «   »)
      Fin Pour

Fin
```

Les chaînes de caractères

Les chaînes de caractères

Définition

Une chaîne de caractères est une séquence de plusieurs caractères ➔ Elle est représentée comme un tableau de caractères

Indices des caractères

1	2	3	4	5	6	7
B	O	N	J	O	U	R

Chaque caractère est accessible séparément

Les chaînes de caractères

Déclaration & Initialisation

Une variable de type chaîne de caractères se déclare comme suit :

nom_chaine : Chaîne de caractères

L'initialisation d'une chaîne de caractères se fait par simple affectation d'une valeur entre « » à la variable

nom_chaine ← valeur

Exemple

```
Algorithme exemple_chaine_caracteres

Variables :
          cours: chaîne de caractères

Début
     cours ← « algorithmique »
Fin
```

Les chaînes de caractères

Saisie & Affichage

Une variable de type chaîne de caractères est saisie et affichée en bloc : Pas besoin de le faire caractère par caractère

```
Algorithme saisie_affichage_chaine_caracteres

Variables :
          cours: chaîne de caractères

Début
     Lire(cours)
     Ecrire (cours)
Fin
```

Les chaînes de caractères

Accès aux éléments d'une chaîne de caractères

L'accès à un caractère d'une variable de type chaîne de caractères se fait par l'indice comme pour un tableau.

nom_chaine [indice]

```
Algorithme acces_caracteres

Variables :
           cours: chaîne de caractères
           lettre : caractère

Début
      cours ← « algorithmique »
      lettre ← cours[1]
      Ecrire (lettre)
Fin
```

À la fin de cet algorithme, la lettre 'A' est affichée à l'écran

```
Algorithme acces_caracteres_inexistant

Variables :
           cours: chaîne de caractères
           lettre : caractère

Début
      cours ← « algorithmique »
      lettre ← cours[1]
      Ecrire (cours[15])
Fin
```

Erreur !!! Le mot ne contient que 13 caractères, le caractère 15 n'existe pas

Les chaînes de caractères

Taille d'une chaîne de caractères

La taille de la chaîne de caractères n'est pas précisée à la déclaration , elle est connue au moment de son initialisation.

Pour avoir la taille d'une variable de type chaîne de caractères, on utilise la fonction prédéfinie longueur() comme suit : variable ← longueur(nom_chaine)

Exemple

```
Algorithme longueur_chaine_caracteres

Variables :
           cours: chaîne de caractères
           taille : entier
Début
      cours ← « algorithmique »
      taille ← longueur(cours)
      Ecrire (taille)

Fin
```

À la fin de cet algorithme, la valeur 13 est affichée à l'écran

Les chaînes de caractères

Parcourir une chaîne de caractères

Comme pour un tableau, il est possible de parcourir une chaîne de caractères en utilisant une boucle.

Exemple

```
Algorithme parcours_complet_chaine_caracteres

Variables :
          cours: chaîne de caractères
          i: entier (compteur d'itérations)
Début
     Lire(cours)
     Pour i ← 1 À longueur(cours) Faire
          Ecrire (cours[i], « \NL »)
     Fin Pour
Fin
```

À la fin de cet algorithme, chacune des lettres de la variable « cours » sera affichée sur une ligne

Les chaînes de caractères

Chaîne de caractères Vs Tableau 1D

	Tableau 1D	Chaîne de caractères
Déclaration	Avec le mot clé Tableau de	Avec le type chaîne de caractères
Saisie	Case par case dans une boucle	En bloc
Affichage	Case par case dans une boucle	En bloc
Accès aux éléments	Par l'indice	Par l'indice
Taille	1 ≤ Taille logique ≤ Taille Physique	Obtenue par la fonction longueur()
Parcours	En utilisant une boucle	En utilisant une boucle

Les tableaux 2D

efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

Les tableaux 2D

Motivation

Supposons que l'on veuille représenter un sudoku basique avec les outils algorithmiques dont on dispose jusque-là. Comment faire ?

- Utiliser une variable ? De quel type ? ➔ Ce n'est pas possible , nous avons plusieurs valeurs !!!
- Utiliser un tableau 1D ? Il peut regrouper toutes les valeurs MAIS ne permet pas de distinguer les différents niveaux (les lignes, les colonnes) !!!

1	2	3
6		4
7	8	9

Utiliser un tableau où l'on rangera dans chaque case une rangée (ligne) du Sudoku.

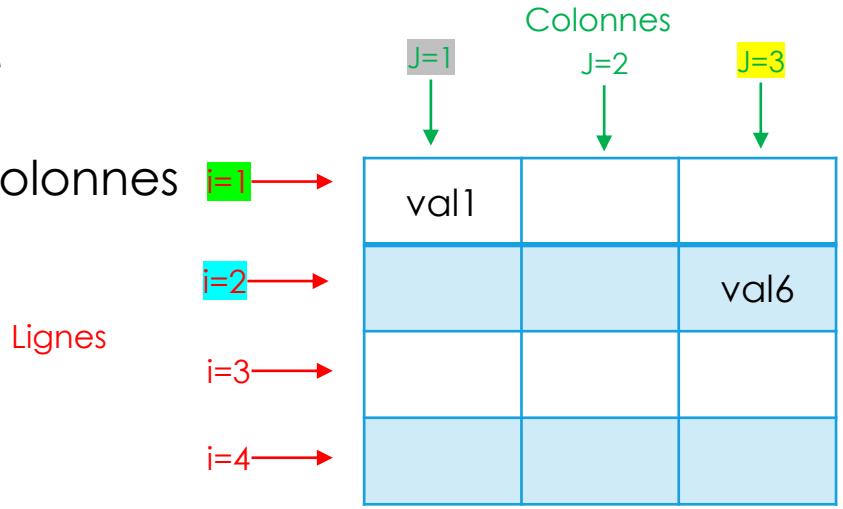
1 2 3	6 4	7 8 9

Chaque case du Grand tableau contient à son tour un tableau 1D ➔ Nous avons un tableau de tableaux.

Les tableaux 2D

Définition

- Un tableau à 2D est un tableau où les valeurs des cases sont, à leur tour, de type tableau 1D de même taille.
- Pour faciliter sa représentation , L'on préfère le voir comme étant une matrice caractérisée par:
  - Des lignes et des colonnes et dont toutes les valeurs sont de même type.
  - Les lignes ont des indices allant de 1 à NB_Ligne
  - Les colonnes ont des indices allant de 1 à NB_Colonnes



Les tableaux 2D

Déclaration

- Un tableau à 2D ou matrice se déclare en algorithmique comme suit :

nom_matrice[nombre_lignes][nombre_colonnes] : Tableau de Type

Nombre maximum de cases mémoire réservées pour les lignes: taille physique

Nombre maximum de cases mémoire réservées pour les colonnes: taille physique

Type des valeurs à stocker dans la matrice

Exemple

```
Algorithme Exemple_declaration_matrice


Variables :
          mat1[20][30] : Tableau de réel
          damier[8][8] : Tableau de entier
Début
     ...
Fin
```

Les tableaux 2D

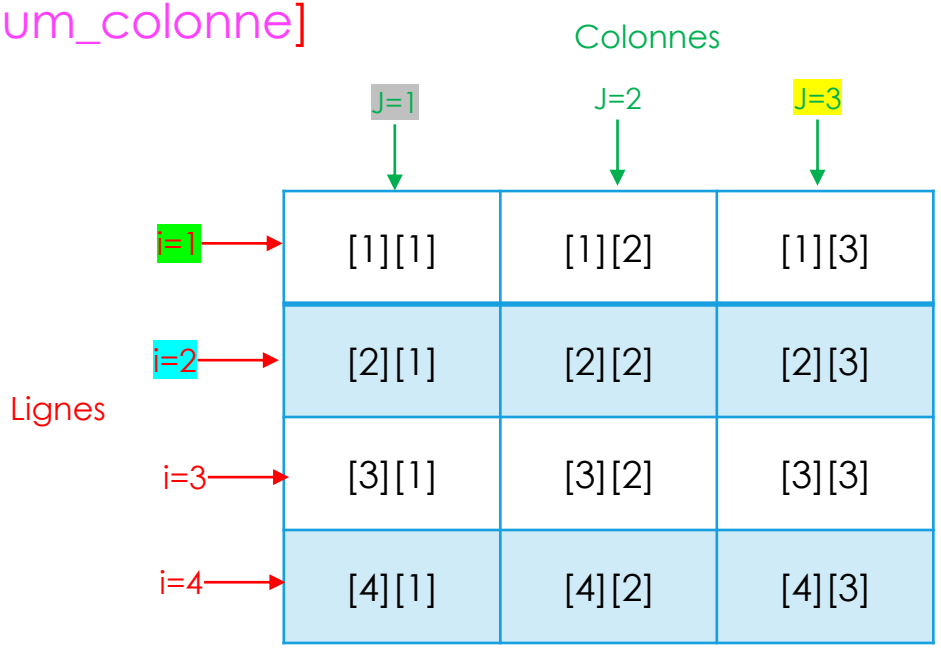
Accès aux valeurs

Pour accéder à la valeur d'une case, l'indice de ligne est croisé avec l'indice de la colonne où elle se trouve ➔ 2 indices

nom_matrice[num_ligne][num_colonne]

```
Algorithme Exemple_acces_matrice

Variables :
            damier[8][8] : Tableau de entier
Début
      damier[1][1] ← 0
      damier[1][2] ← 1
Fin
```



Les tableaux 2D

Initialisation d'une matrice

Pour initialiser un tableau 2D en dur, il suffit de lui affecter les valeurs comme suit:

nom-matrice ← { {valeurs ligne 1},
{valeurs ligne 2},
....
}

Exemple

```
Algorithme Exemple_initialisation_matrice

Variables :
          mat[2][3]: Tableau de entier

Début
     mat ← {{1, 2, -4},
            {78, 34, 3}
            }
Fin
```

Équivalent à

mat[1][1] ← 1
mat[1][2] ← 2
mat[1][3] ← -4
mat[2][1] ← 78
mat[2][2] ← 34
mat[2][3] ← 3

Les tableaux 2D

Saisies des valeurs

- Pour remplir toutes les cases d'une matrice, il faut:
  1. Demander à l'utilisateur ou avoir les tailles logiques correspondantes aux nombres de lignes et nombre de colonnes (cela définira la quantité exacte de valeurs stocker)
  2. Utiliser 2 boucles « Pour » imbriquées: une pour les lignes, une pour les colonnes.

Exemple

```
Algorithme Exemple_saisir_matrice
Constantes :
            NB_LIGNES = 8
            NB_COLONNES = 10
Variables :
            mat[NB_LIGNES][NB_COLONNES] : Tableau de entier
            nb_lig, nb_col, i, j: entier
Début
      Répéter
            Ecrire (« Donner le nombre de lignes »)
            Lire(nb_lig)
      Tant que (nb_lig ≤ 0 OU nb_lig > NB_LIGNES )
      Répéter
            Ecrire (« Donner le nombre de colonnes »)
            Lire(nb_col)
      Tant que (nb_col ≤ 0 OU nb_col > NB_COLONNES )
      Pour i ← 1 À nb_lig Faire
            Pour j ← 1 À nb_col Faire
                  Ecrire (« Saisir une valeur »)
                  Lire(mat[i][j])
            Fin Pour
      Fin Pour
Fin
```

Saisie sécurisée pour le nombre de lignes

Saisie sécurisée pour le nombre de colonnes

Remplissage de la matrice

Les tableaux 2D

Affichage des valeurs

- L'affichage des éléments d'une matrice se fait valeur par valeur.
- Pour pouvoir parcourir toute la matrice et afficher toutes ses valeurs, il faut utiliser 2 boucles « Pour » imbriquées: une pour les lignes, une pour les colonnes.

Exemple

```
Algorithme Exemple_saisir_matrice
Constantes :
            NB_LIGNES = 8
            NB_COLONNES = 10
Variables :
            mat[NB_LIGNES][NB_COLONNES] : Tableau de entier
            nb_lig, nb_col, i, j: entier
Début
      Répéter
            Ecrire (« Donner le nombre de lignes »)
            Lire(nb_lig)
      Tant que (nb_lig ≤ 0 OU nb_lig > NB_LIGNES )
      Répéter
            Ecrire (« Donner le nombre de colonnes »)
            Lire(nb_col)
      Tant que (nb_col ≤ 0 OU nb_col > NB_COLONNES )
      Pour i ← 1 À nb_lig Faire
            Pour j ← 1 À nb_col Faire
                  Ecrire (« Saisir une valeur »)
                  Lire(mat[i][j])
            Fin Pour
      Fin Pour
      Pour i ← 1 À nb_lig Faire
            Pour j ← 1 À nb_col Faire
                  Ecrire (mat[i][j], «  »)
            Fin Pour
            Ecrire (« \NL »)
      Fin Pour
Fin
```

Affichage complet de la matrice : Ne pas oublier le retour à la ligne pour que les valeurs ne soient pas toutes affichées sur une même ligne